

Capítulo 23

Invocación remota

Al terminar este capítulo, entenderás:

- Qué es un protocolo de petición-respuesta.
- Cuáles son las distintas semánticas de invocación.
- Qué es y cómo funciona la invocación remota.
- Qué son RPC y RMI.

El capítulo 15 presentó el patrón petición-respuesta: el cliente envía una petición —normalmente una operación sobre un recurso— y el servidor responde con un resultado. Ese patrón lo define, en cada caso, un protocolo de aplicación propio: HTTP para la web, SMTP para el correo, DNS para la resolución de nombres, etc. Cada uno especifica su propio formato de mensajes, sus propias operaciones y su propia forma de indicar errores, adaptados al problema concreto que resuelven.

Diseñar un protocolo de aplicación desde cero no es trivial: hay que definir la sintaxis de los mensajes, decidir cómo se delimitan sobre el flujo de bytes subyacente, considerar la evolución del protocolo (versiones, compatibilidad) y, finalmente, implementar esa especificación en cada lenguaje o plataforma que vaya a usarla. Para muchas aplicaciones, ese coste es desproporcionado en relación con lo que realmente necesitan: simplemente ejecutar código en un nodo remoto y obtener el resultado, igual que se invoca una función o un método local.

La **invocación remota** es una solución general para esa necesidad. En lugar de definir un protocolo específico por aplicación, ofrece al programador una infraestructura genérica —el **middleware** de comunicaciones—, común a cualquier aplicación, que se encarga de transportar la invocación hasta el nodo remoto, ejecutarla allí y devolver el resultado. RPC (Remote Procedure Call) sigue este enfoque para procedimientos (funciones independientes); RMI (Remote Method Invocation) hace lo mismo para métodos de objetos. En ambos casos, el middleware genera el código encargado de

establecer la comunicación, serializar los argumentos, enviarlos, ejecutar el código, devolver la respuesta y entregar el resultado de vuelta al cliente, de modo que invocar una operación remota se parece, desde el punto de vista del programador, a invocar una operación local.

Lo que realmente diferencia una invocación remota de un mero intercambio de mensajes es que la invocación implica la ejecución de un fragmento de código en el receptor a expensas del emisor. La petición indica qué pieza de código se ha de ejecutar y con qué argumentos, mientras que la respuesta transporta el resultado de dicha ejecución.

En la invocación remota, el que envía el mensaje de petición (la invocación) es el cliente, y el que recibe y ejecuta el código asociado es el servidor. Sin embargo, aunque es lo habitual, esos roles no tienen porqué coincidir con los de los procesos involucrados, es decir, un servidor TCP podría ser el que envía la invocación hacia el cliente TCP.

23.1. Protocolos de petición-respuesta

Dos procesos que se ejecutan en nodos distintos no comparten memoria, así que la única forma de interactuar es enviándose mensajes a través de la red: eso es el *paso de mensajes*. Se reduce, en su forma más básica, a dos primitivas sobre un socket, `send(destino, mensaje)` y `receive(origen, mensaje)`, sin que deba existir ninguna relación entre mensajes sucesivos.

El patrón *petición-respuesta* es la forma más simple de paso de mensajes en una comunicación extremo a extremo: asocia un mensaje de respuesta al de petición petición que la originó. Este patrón Lo emplean protocolos de varias capas, pero es especialmente frecuente en la de aplicación.

Diseñar un protocolo de aplicación obliga al diseñador a tomar esencialmente las mismas decisiones, independientemente del problema que resuelva el protocolo, aunque el resultado de esas decisiones varía según las necesidades de la aplicación.

La invocación remota no es diferente, aunque no resuelve todas esas decisiones del mismo modo: el middleware fija de antemano algunas de ellas al generar automáticamente el protocolo subyacente, mientras que deja otras en manos del programador. Estas son algunas:

- **Síncrono vs. asíncrono:** el cliente de la petición puede bloquear en espera de la respuesta, o continuar inmediatamente con otra tarea. La primera es una invocación síncrona y es más fácil de entender, porque el cliente sabe en todo momento si la petición sigue pendiente; además es más similar a una llamada local. Una invocación asíncrona

aprovecha el tiempo de espera, pero debe encargarse de correlacionar más adelante cada respuesta con la petición que le corresponda.

- **Con o sin estado:** el servidor puede mantener el estado entre peticiones sucesivas del mismo cliente, o resolver cada una de forma independiente, sin memoria de las anteriores. Un protocolo sin estado es más sencillo de escalar y de recuperar tras un fallo, porque una réplica del servicio puede atender la siguiente petición sin compartir contexto previo con la que atendió la anterior; a cambio, cualquier información que dependa de peticiones anteriores tiene que viajar explícitamente en cada una.
- **Transporte y fiabilidad:** sobre TCP la conexión ya garantiza entrega y orden, pero establecerla tiene un coste que no siempre compensa para un intercambio breve. Sobre UDP no hay tal garantía: si se pierde el mensaje con la petición o el que lleva la respuesta, nada lo detecta ni corrige.

Entre la fiabilidad completa que ofrece TCP y la ausencia total de UDP hay un término medio (o varios): el protocolo de petición-respuesta puede construir sobre UDP solo las medidas de fiabilidad que necesite, sin la sobrecarga que implican las que no necesita. El identificador de mensaje, como el que llevan las consultas DNS, permite cotejar cada respuesta con la petición que la originó, aunque haya varias en curso o el transporte las reordene; ese mismo identificador le permite al servidor detectar y descartar duplicados en lugar de procesarlos de nuevo. Un temporizador permite detectar peticiones o respuestas perdidas. Con ese mismo temporizador se puede añadir la retransmisión, reintentando la petición un número limitado de veces antes de darla por fallida definitivamente. Identificador, temporizador y retransmisión son las piezas básicas de cualquier protocolo ARQ; cada protocolo de petición-respuesta elige cuáles de ellas necesita, y con qué grado de exigencia.

Estos elementos definen tres estilos de intercambio, según qué mensajes intervienen y con qué fin:

- **R (Request):** el cliente envía la petición y no espera ninguna respuesta (no bloqueante). Sirve para peticiones *one-way* habitualmente sobre transporte no confiable. No hay un valor de retorno, confirmación, timeouts ni retransmisiones. También podría ocurrir que hubiera un resultado o confirmación, pero al cliente de la petición no le interese, por lo que no espera ni recoge el mensaje.

- **RR (Request-Reply):** El cliente espera un mensaje de respuesta (bloqueante). La respuesta, además de potencialmente contener un resultado, puede llevar un código de confirmación o de error. Puede utilizar TCP como transporte o bien implementar confiabilidad básica con un ARQ. RR es el estilo más común en protocolos de aplicación.
- **RRA (Request-Reply-Acknowledge):** cuando el servidor detecta una petición duplicada, no basta con descartarla: si ya ejecutó la operación, tiene que reenviar el mismo resultado en lugar de ejecutarla de nuevo, así que necesita conservarlo en una caché llamada *historia*, indexada por el identificador de mensaje, en lugar de descartarlo justo después de enviarlo. La duda es cuánto tiempo mantener cada entrada: sin ninguna señal del cliente, el servidor solo puede adivinarlo con un temporizador propio y un margen de seguridad. Este estilo añade un tercer mensaje con el que el cliente confirma la recepción de la respuesta, y con el que el servidor sabe explícitamente cuándo puede liberar la entrada de la historia. Sin este mensaje, la historia crecería indefinidamente.

Elegir el estilo de intercambio es una de las decisiones que puede tomar el middleware, mientras que otras —como la elección síncrono/asíncrono— son delegadas al programador. Eso no es algo negativo, sino que precisamente le otorga más control en relación a las necesidades y requisitos de la aplicación.

23.2. Medidas de tolerancia a fallos

Frente a la pérdida de un mensaje, la invocación remota puede aplicar tres medidas, que ofrecen fiabilidad creciente:

- **Reintentar la petición:** si el temporizador expira sin recibir respuesta, el cliente retransmite el mismo mensaje, con el mismo identificador. Se aplica un número limitado de repeticiones, que de alcanzarse, será considerado un error. Obviamente implica obligatoriamente un mensaje de respuesta/confirmación. Sin esta medida, el cliente no puede saber si la petición llegó al servidor.
- **Filtrar duplicados**¹: el servidor conserva los identificadores de las peticiones ya atendidas, para reconocer una retransmisión y no repetir la ejecución de la operación que le corresponde. Sin este filtro, cada retransmisión se procesa de nuevo.

¹denominado a menudo «deduplicación»

- **Retransmisión del resultado:** al detectar una petición duplicada, el servidor no se limita a descartarla, sino que reenvía el resultado que ya calculó y tiene guardado, en lugar de dejar al cliente sin respuesta pese a que la operación sí llegó a ejecutarse.

23.3. Operaciones idempotentes

Una operación es **idempotente** cuando ejecutarla varias veces (con los mismos argumentos) produce sobre el estado del servidor el mismo efecto que ejecutarla una sola vez. La idempotencia es una propiedad de la semántica propia de la operación. Es interesante porque estas operaciones no requieren detección de duplicados, lo que simplifica su implementación.

Un estilo RRA necesita cachear la respuesta de cada petición precisamente en el caso de las operaciones no idempotentes, que normalmente se asume que es caso más común.

Algunos ejemplos de operaciones idempotentes podrían ser:

```
set_size(25)
stop()
car.get_color()
get_time()
```

Es interesante destacar que `get_time()` sea idempotente, porque aunque el valor que retorne sea diferente en cada llamada, cumple la condición de no modificar el estado del servidor.

Mientras que los siguientes son no idempotentes:

```
push("item")
counter.inc()
send_mail("user@example.com", "Hello")
```

Este concepto de idempotencia no coincide con el que se utiliza en matemáticas. Aquel está relacionado con el resultado de la aplicación de una función al resultado previo de esa misma función, que de hecho ni siquiera aplica a varios de los ejemplos anteriores porque requiere que la función tenga al menos un argumento.

23.4. Semánticas de invocación

Combinar las tres medidas de tolerancia a fallos —reintentar petición, filtrar duplicados y retransmitir resultado— da lugar a distintas garantías sobre potencialmente cuántas veces se llega a ejecutar la operación remota. Estas garantías se conocen como semánticas de invocación:

- *maybe*: no aplica ninguna medida. Si se pierde la petición o la respuesta, el cliente no tiene forma de saberlo ni de corregirlo. La operación se ejecuta cero o una vez, sin que el cliente pueda distinguir cuál de los dos casos ha ocurrido. Es la semántica que resulta, por ejemplo, de un estilo R (Request) sobre un transporte no confiable. Por ejemplo es útil cuando el cliente envía mensajes actualizando periódicamente el valor de una medida (*p. ej.* la temperatura de una habitación), y no es un problema que se pierda alguno de esos mensajes.
- *at-least-once*: reintenta la petición hasta tener una confirmación, pero no filtra duplicados. Cada retransmisión que llega al servidor se ejecuta de nuevo, así que la operación puede llegar a ejecutarse una o más veces. Garantiza que se ejecuta al menos una vez, a costa de no poder garantizar que no se repita. Se puede implementar con el estilo RR o R con transporte confiable si no requiere valor de retorno.
- *at-most-once*: además de reintentar, el servidor filtra los duplicados. Conserva el resultado ya calculado de la primera ejecución para volver a enviarlo si la petición se repite. La operación se ejecuta como mucho una vez (0 o 1 veces). El cliente siempre recibe una respuesta, ya sea para informar del resultado o de un error, es decir, a diferencia de *maybe* el cliente sabe si la operación se ejecutó o no. Es la semántica que ofrece el estilo RRA (Request-Reply-Acknowledge).
- *exactly-once*: ejecuta la operación exactamente una vez. Esto es algo que las tres medidas de tolerancia a fallos no pueden garantizar por sí solas, porque únicamente actúan sobre la comunicación. La garantía *exactly-once* requiere asegurar la continuidad del estado del servidor, particularmente entre la ejecución de la operación y su registro en la historia, por lo que ambas se ejecutan como una operación atómica. Además requiere de un sistema adicional para recuperar al proceso servidor tras un *crash* con recuperación transparente del estado previo. Eso lo convierte en un problema realmente complejo y difícilmente justificable. En la práctica, lo que se suele implementar es una variante más simple llamada *effectively-once*, que es una combinación de *at-least-once* sobre operaciones idempotentes.

23.5. Mecanismo de invocación remota

Con independencia de la tecnología concreta —RPC, RMI u otra—, todo mecanismo de invocación remota comparte una arquitectura similar. Comienza con una *interfaz remota* conocida tanto por el cliente como por el servidor. La interfaz es una lista de las operaciones que ofrecerá (implemen-

tará) el servidor, con sus nombres, parámetros y tipos de retorno. mientras que el cliente será quien invoque esas operaciones.

El middleware proporciona habitualmente un compilador que genera el código necesario para transportar las invocaciones (identificadores y argumentos) hasta el servidor y llevar los resultados de vuelta al cliente.

Esos fragmentos de código se denominan *stubs* (cabos) y realizan tareas diferentes para cliente y el servidor. Los *stubs* del cliente proporcionan funciones que implementan la interfaz remota, pero no tienen funcionalidad de la aplicación. Lo que hacen es serializar y transportar las invocaciones hasta el servidor. Los *stubs* del servidor incluyen la lógica de multiplexación que localiza y ejecuta funciones con las mismas firmas. Es habitual que ese compilador ofrezca la posibilidad de generar implementaciones huecas para las operaciones, que suelen denominarse *esqueletos* (*skeletons*).

El recorrido completo de una invocación remota, desde que el cliente invoca su *stub* hasta que recibe el resultado, sigue siempre estos pasos:

Cliente	Servidor
El código del cliente invoca la operación remota pasando los argumentos correspondientes, exactamente igual que invocaría una función o un método local, pero en realidad está invocando al <i>stub</i> .	
El <i>stub</i> serializa el identificador de la operación solicitada junto con los argumentos recibidos, y con esa información construye el mensaje de petición.	
El <i>stub</i> del cliente envía la petición al nodo servidor, con el estilo de intercambio y las medidas de fiabilidad aplicados.	
	El <i>stub</i> del servidor recibe la petición. El identificador de operación incluido en el mensaje indica qué procedimiento o método se ha de ejecutar.
	El <i>stub</i> del servidor deserializa los argumentos contenidos en el mensaje, reconstruyendo a partir de ellos los valores nativos que necesita la operación.
	El <i>stub</i> del servidor invoca localmente la implementación real de la operación.
	Cuando la ejecución termina, el <i>stub</i> del servidor serializa el resultado —o la información del error, si la operación falló— en un mensaje de respuesta.
	El <i>runtime</i> del servidor envía la respuesta.
El <i>runtime</i> del cliente la recibe y la entrega al <i>stub</i> .	
El <i>stub</i> deserializa el resultado y lo devuelve como valor de retorno de la llamada original.	

El identificador de la operación (paso 4) le indica al servidor qué operación quiere ejecutar. No se debe confundir con el identificador de mensaje (§23.1), que se utiliza para distinguir ocurrencias distintas, incluso aunque correspondan a la misma operación.

El mecanismo de invocación remota esencialmente ofrece transparencia: la invocación al método remoto es muy similar sintácticamente a una llamada local equivalente. Sin embargo, hay varias diferencias que no se pueden (y no se deben) ocultar: latencia, errores en el servidor, fallo parcial o las garantías de entrega aplicadas (§23.4).

23.5.1. pico-rpc

Pico-rpc es un middleware RPC mínimo escrito en Python y con fines exclusivamente docentes. Implementa las piezas elementales del mecanismo de invocación remota: lenguaje de descripción de interfaces, compilador, generación de los *stubs* de cliente y servidor, y *dispatcher*; pero no implementa la mayoría del control de errores o mecanismos de seguridad. Con pico-rpc resulta sencillo hacer el recorrido completo de la descripción de RPC de COULOURIS *et al.* [63]. Consta de los siguientes ficheros:

- `picorpc.py` es el *runtime* del middleware. Contiene las primitivas para describir interfaces y el *dispatcher* del servidor.
- `rpcgen.py` es el compilador de interfaces. Genera los *stubs* a partir de la definición de la interfaz.
- `math.idl.py` es la definición de la interfaz del ejemplo.
- `server.py` corresponde con el servidor de la aplicación, que incluye implementación de los procedimientos remotos que aporta el programador de la aplicación. Por último `client.py` es el cliente que realiza la invocación.

Interfaz

Normalmente, la definición de la interfaz se escribe con un lenguaje específico y puramente declarativo, llamado IDL, que es independiente del lenguaje de programación en el que se implementa cliente y servidor. El compilador genera los *stubs* en el lenguaje de destino a partir de esa definición. Por simplicidad, en pico-rpc, la definición de la interfaz se escribe también en Python mediante una clase que hereda de `Interface`, y un método sin cuerpo por cada operación. Las anotaciones de tipo de los parámetros y del valor de retorno determinan cómo se serializa cada valor (Listado 23.1).

```
from picorpc import Interface, uint8, uint64

class Math(Interface):
    def factorial(n: uint8) -> uint64: ...
    def power(base: uint8, exp: uint8) -> uint64: ...
```

LISTADO 23.1: Interfaz del servicio Math

📄/pico-rpc/math.idl.py

Los tipos `uint8` y `uint64` corresponden directamente con los caracteres de formato de `struct` para enteros sin signo de 1 y 8 bytes ('B' y 'Q'). El identificador de cada operación es su posición en la interfaz: 0 para `factorial()` y 1 para `power()`.

El «compilador» lee esta definición y genera los *stubs* de cliente y servidor:

```
$ ./rpcgen.py math.idl.py
generated math_client_stub.py and math_server_stub.py from math.idl.py
```

Cliente

El *stub* de cliente es una clase (`MathStub`) con un método por cada operación de la interfaz (Listado 23.2). Cada método abre una nueva conexión TCP, construye el mensaje de petición con el identificador de la operación (1 byte) seguido de los argumentos serializados, y espera la respuesta, que contiene el resultado serializado como un entero de 8 bytes. Esto encaja con el estilo RR en el que cada conexión transporta una única petición y su respuesta. No hace falta un identificador de mensaje para asociarlas, porque la conexión no se reutiliza y TCP resuelve las retransmisiones de segmentos sin que la aplicación reciba duplicados.

```
class MathStub:
    FACTORIAL = 0
    POWER = 1

    def __init__(self, host, port):
        self.server_endpoint = host, port

    def factorial(self, n):
        sock = socket.socket()
        sock.connect(self.server_endpoint)

        request = struct.pack('BB', self.FACTORIAL, n)
        sock.sendall(request)

        reply = sock.recv(MAX_MESSAGE_SIZE)
        result = struct.unpack('Q', reply)[0]

        sock.close()
        return result
```

LISTADO 23.2: *Stub* de cliente generado (fragmento)
`math_client_stub.py`

El programa cliente solo necesita instanciar el *stub* indicando el *endpoint* del servidor e invocar sus métodos del mismo modo que lo haría con un objeto convencional (Listado 23.3).

```
import sys

from math_client_stub import MathStub

if len(sys.argv) != 3:
    print("usage: ./client <server> <arg>")
```

```

exit(1)

server = sys.argv[1]
arg = int(sys.argv[2])

stub = MathStub(server, 2000)
print("factorial({}) = {}".format(arg, stub.factorial(arg)))
print("power({}, {}) = {}".format(arg, arg-1, stub.power(arg, arg-1)))

```

LISTADO 23.3: Cliente del servicio Math

[📄/pico-rpc/client.py](#)

Servidor

En el lado del servidor, el compilador genera una función *stub* por cada operación y una tabla (STUBS) que asocia cada identificador de operación con su función *stub* (Listado 23.4). Cada una de esas funciones deserializa los parámetros, invoca la implementación real de la operación y serializa el resultado.

```

def factorial_stub(args, implementation):
    args = struct.unpack('B', args)
    result = implementation.factorial(*args)
    return struct.pack('Q', result)

def power_stub(args, implementation):
    args = struct.unpack('BB', args)
    result = implementation.power(*args)
    return struct.pack('Q', result)

FACTORIAL = 0
POWER = 1

STUBS = {
    FACTORIAL: factorial_stub,
    POWER: power_stub,
}

```

LISTADO 23.4: *Stub* de servidor generado[math_server_stub.py](#)

La multiplexación la realiza el *dispatcher* (clase `Dispatcher` de `picorpc.py`). Acepta conexiones de forma iterativa (como en § 15.2), lee el mensaje de petición, utiliza su primer byte para localizar la función *stub* correspondiente en la tabla STUBS y le entrega el resto del mensaje, es decir, los argumentos serializados. Por último, envía al cliente el resultado que devuelve el *stub*.

En el servidor el programador define una clase que implementa las operaciones de la interfaz (`MathI`), y registra una instancia en el *dispatcher* junto con el módulo del *stub* de servidor (Listado 23.5).

```

from picorpc import Dispatcher
import math_server_stub

class MathI:
    def factorial(self, n):
        if n == 0:
            return 1

        return n * self.factorial(n - 1)

    def power(self, base, exp):
        return base ** exp

dispatcher = Dispatcher(port=2000)
dispatcher.register(math_server_stub, MathI())
dispatcher.run()

```

LISTADO 23.5: Servidor del servicio Math
 /pico-rpc/server.py

Es importante aclarar que a pesar de que pico-rpc utiliza clases tanto para la definición de la interfaz y para la implementación de las operaciones, lo hace solo por conveniencia de implementación. Sus métodos no tienen relación entre sí y funcionan como operaciones independientes.

Ejecución del servidor y del cliente en el mismo computador:

Cliente

```

$ ./client.py 127.0.0.1 5
factorial(5) = '120'
power(5, 4) = '625'

```

Servidor

```

$ ./server.py
Server ready: ('0.0.0.0', 2000)

```

FIGURA 23.1: Servidor y cliente pico-rpc

Para mantener el código al mínimo, pico-rpc prescinde deliberadamente de aspectos que ningún middleware real puede ignorar. Solo admite enteros sin signo, supone que cada mensaje llega completo en una única llamada a `recv()` —ignora la E/S parcial— el servidor es iterativo y no aplica ninguna medida de tolerancia a fallos más allá de las que proporciona TCP. Tampoco transporta errores de vuelta al cliente si la ejecución de la operación o la serialización falla, *p. ej.* el resultado de `factorial(21)` no cabe en un `uint64`.

23.5.2. ONC RPC

ONC RPC—también llamado Sun RPC— fue uno de los middlewares pioneros de la invocación remota. Lo desarrolló Sun Microsystems en los años

80 y sobre él está construido el popular sistema de ficheros en red NFS que aún hoy día se sigue utilizando.

Serializa los datos con un formato llamado XDR [64], y las interfaces de los procedimientos remotos se describen en ficheros `.x`, un lenguaje que extiende XDR para declarar programas y procedimientos. La implementación original de Sun, de la que derivan las actuales, solo permite escribir clientes y servidores en C, aunque existen implementaciones independientes para otros lenguajes.

El ejemplo `/onc-rpc-factorial` es un servicio equivalente al de `pico-rpc`, aunque únicamente con la operación `factorial()`. La interfaz (Listado 23.6) declara un «programa», identificado por un número (600000002), con una versión (1) que contiene el procedimiento `FACTORIAL`, también numerado (1). El número de procedimiento cumple la misma función que el identificador de operación de `pico-rpc`, pero también en otros middlewares reales.

```
program FACTORIAL_PROG {
    version FACTORIAL_VERS {
        unsigned long FACTORIAL(int n) = 1;
    } = 1;
} = 600000002;
```

LISTADO 23.6: Interfaz del servicio `FACTORIAL_PROG`
`/onc-rpc-factorial/factorial.x`

A partir de la interfaz, `rpcgen` (`rpcsvc-proto`) genera tres ficheros: `factorial.h`, con las constantes y los prototipos; `factorial_clnt.c`, el *stub* de cliente, y `factorial_svc.c`, el *stub* de servidor, que incluye el *dispatcher* y la función `main()` del servidor. El programador aporta lo demás:

- `factorial_server.c` es la implementación del procedimiento en la función `factorial_1_svc()`, cuyo nombre se forma con el del procedimiento y el número de versión. Devuelve un puntero a una variable estática porque el *stub* serializa el resultado después de que la función haya retornado (Listado 23.7).
- `factorial_client.c` obtiene un manejador de cliente con `clnt_create()`, indicando el nodo, el programa, la versión y el transporte (UDP), e invoca la operación con `factorial_1()`, que es la función del *stub* de cliente (Listado 23.8).

```
u_long *factorial_1_svc(int *n, struct svc_req *rqstp) {
    static u_long result;
    result = 1;

    printf("Received: n=%d \n", *n);
```

```

    for (int i=*n; i>0; i--)
        result *= i;

    return &result;
}

```

LISTADO 23.7: Implementación del procedimiento FACTORIAL
 /onc-rpc-factorial/factorial_server.c

```

void factorial_prog_1(char *host, int n) {
    CLIENT *clnt;
    u_long *result_1;

#ifdef DEBUG
    clnt = clnt_create (host, FACTORIAL_PROG, FACTORIAL_VERS, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif

    result_1 = factorial_1(&n, clnt);
    if (result_1 == NULL)
        clnt_perror(clnt, "call failed");
    else
        printf("factorial(%d): %lu", n, *result_1);

#ifdef DEBUG
    clnt_destroy (clnt);
#endif
}

```

LISTADO 23.8: Cliente del servicio FACTORIAL_PROG
 /onc-rpc-factorial/factorial_client.c

Al arrancar, el servidor crea un socket UDP y otro TCP, sin puerto fijo, y los registra en `rpcbind` (versión actual del antiguo «portmapper»), un servicio que asocia cada combinación de programa, versión y transporte con el puerto en el que está disponible.

El servidor ofrece los dos transportes para que el cliente pueda elegir cuál usar. Para eso se utiliza el último argumento de `clnt_create()` (Listado 23.8). Lógicamente cada transporte implica una semántica de invocación distinta (§23.4). La función `clnt_create()` consulta a `rpcbind` en el nodo remoto para averiguar el puerto que corresponde al transporte elegido.

- Con UDP el *runtime* del cliente retransmite la petición si la respuesta no llega a tiempo, y cada mensaje lleva un identificador de transacción (*xid*) para asociar la respuesta con su petición. Sin embargo, el servidor no filtra duplicados salvo que se active expresamente su caché de peticiones, así que la semántica es *at-least-once*. Con la caché

activa, el servidor reenvía la respuesta guardada en lugar de ejecutar de nuevo el procedimiento, lo que se aproxima a *at-most-once*, aunque la caché tiene un tamaño limitado y se pierde si el servidor se reinicia.

- Con TCP el *runtime* del cliente no retransmite. La petición se envía una sola vez y es TCP quien garantiza que llega sin pérdidas ni duplicados. Si la conexión se interrumpe o expira la espera, la llamada devuelve un error sin reintentarlo y el cliente no puede saber si el procedimiento llegó a ejecutarse. El procedimiento se ejecuta, por tanto, como mucho una vez: semántica es *at-most-once*.

El cliente del ejemplo utiliza UDP. Para este servicio es suficiente, porque calcular el factorial es una operación idempotente.

El ejemplo `🐞/onc-rpc-hello` es aún más básico. Solo define un procedimiento que recibe una cadena y la imprime en el servidor. Está tomado, con algunas simplificaciones, de la documentación oficial de ONC RPC [65]. Incluye el mismo programa en dos versiones, una local con una llamada a función convencional (directorio `local`) y otra en la que esa función pasa a ser un procedimiento remoto (directorio `remote`), lo que permite comparar qué cambia al convertir una llamada local en remota.

23.5.3. gRPC

gRPC es un middleware RPC de código abierto que Google publicó en 2015 a partir de su experiencia con su sistema interno de invocación remota. Actualmente es uno de los más utilizados para comunicar servicios dentro de la infraestructura interna o un centro de datos, sobre todo para arquitecturas de microservicios.

Las interfaces se describen en ficheros con extensión `.proto` en el lenguaje Protocol Buffers, que es a la vez el IDL y el formato de serialización binario. A diferencia de ONC RPC, el compilador genera *stubs* para muchos lenguajes (C++, Java, Go, Python, etc.), de modo que cliente y servidor se pueden escribir en lenguajes distintos. Como transporte utiliza HTTP/2 sobre TCP, y opcionalmente acepta cifrado con TLS.

El ejemplo `🐞/grpc-factorial` es equivalente al ejemplo de ONC RPC, pero en este caso se puede escribir en Python. La interfaz (Listado 23.9) declara un «servicio» `Math` con la operación `factorial()`. Una operación gRPC recibe exactamente un mensaje y devuelve exactamente otro, por lo que el parámetro y el resultado se definen como mensajes (`FactorialRequest` y `FactorialReply`), aunque cada uno contenga un único campo. El número que acompaña a cada campo (= 1) es el que lo identifica en el mensaje serializado. Eso permite añadir campos nuevos a un mensaje sin romper la

compatibilidad con clientes o servidores que usan la versión anterior de la interfaz, algo que Google consideró una característica esencial en el diseño de Protocol Buffers.

```
syntax = "proto3";

package math;

service Math {
  rpc factorial (FactorialRequest) returns (FactorialReply) {}
}

message FactorialRequest {
  int32 value = 1;
}

message FactorialReply {
  int64 result = 1;
}
```

LISTADO 23.9: Interfaz del servicio Math en gRPC
📄/grpc-factorial/math.proto

El compilador `protoc`, que para Python se ejecuta como el módulo `grpc_tools` (■ `python3-grpc-tools`), genera dos ficheros a partir de la interfaz: `math_pb2.py`, con las clases de los mensajes y su serialización, y `math_pb2_grpc.py`, con el *stubs* de cliente (`MathStub`) y servidor (`MathServicer`). El programador aporta lo demás:

- `server.py` implementa la operación en una clase que hereda de `MathServicer` (Listado 23.10). El método `factorial()` recibe el mensaje de petición ya deserializado y un contexto, que da acceso a los metadatos de la llamada y permite, entre otras cosas, devolver un código de error. La función `add_MathServicer_to_server()` registra esa implementación en el servidor, que la asocia con las operaciones de la interfaz. Es el equivalente a la tabla `STUBS` de `pico-rpc`.
- `client.py` crea un canal hacia el *endpoint* del servidor, instancia el *stub* sobre ese canal e invoca la operación con un objeto `FactorialRequest` como argumento (Listado 23.11).

```
import grpc
import math_pb2
import math_pb2_grpc

class Math(math_pb2_grpc.MathServicer):
    def factorial(self, request, context):
        result = 1
        n = request.value
```

```

        while n:
            result *= n
            n -= 1

        return math_pb2.FactorialReply(result=result)

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
math_pb2_grpc.add_MathServicer_to_server(Math(), server)
server.add_insecure_port('[::]:2000')
server.start()

try:
    server.wait_for_termination()

except KeyboardInterrupt:
    server.stop(0)

```

LISTADO 23.10: Servidor del servicio Math en gRPC
 /grpc-factorial/server.py

```

import grpc
import math_pb2
import math_pb2_grpc

if len(sys.argv) != 3:
    print("usage: ./client <server> <value>")
    sys.exit(1)

server = sys.argv[1]
value = int(sys.argv[2])

channel = grpc.insecure_channel('{}:2000'.format(server))
stub = math_pb2_grpc.MathStub(channel)
response = stub.factorial(math_pb2.FactorialRequest(value=value))
print("result = {}".format(response.result))

```

LISTADO 23.11: Cliente del servicio Math en gRPC
 /grpc-factorial/client.py

El servidor escucha en un puerto TCP indicado (2000). No existe ningún equivalente a `rpcbind`, el cliente debe conocer de antemano el `IP` del servidor. Tanto `insecure_channel()` en el cliente como `add_insecure_port()` en el servidor indican que la comunicación no está cifrada. En un despliegue real se aconseja usar sus equivalentes con TLS. Además, a diferencia del de `pico-rpc`, el servidor es concurrente; atiende las peticiones con un *pool* de hasta 10 hilos (`ThreadPoolExecutor`).

Cada invocación viaja como una petición `POST` de HTTP/2 dirigida a una ruta formada por el nombre del servicio y el de la operación (`/math.Math/factorial`). El cuerpo contiene el mensaje serializado precedido de 5 bytes: uno que indica si va comprimido y cuatro con su longitud. El resultado de la ejecución

—éxito o un código de error— viaja en las cabeceras finales de la respuesta. HTTP/2 permite multiplexar varias invocaciones simultáneas sobre la misma conexión TCP, así que el cliente no necesita abrir una conexión por cada llamada, como hace pico-rpc.

Respecto a la semántica de invocación, el *runtime* del cliente no reintenta por defecto una petición que haya podido llegar al servidor, y TCP evita pérdidas y duplicados. Si la llamada falla o expira su plazo, el cliente recibe un error y no puede saber si la operación llegó a ejecutarse, de modo que, igual que ONC RPC sobre TCP, la semántica es *at-most-once*. Los reintentos automáticos se pueden activar, pero en ese caso es responsabilidad del programador determinar si la operación es idempotente.

Por supuesto, gRPC transporta los errores hasta el cliente. Si la implementación de `factorial()` lanza una excepción —*p. ej.* porque el resultado de `factorial(21)` no cabe en el campo `int64` de `FactorialReply`— el cliente recibe un error con el código de estado `UNKNOWN` y el texto de la excepción, en lugar de quedarse esperando o recibir un mensaje corrupto.

Por último, además de las invocaciones «unarias» como la del ejemplo (una petición y una respuesta), gRPC admite otras tres modalidades en las que uno o ambos extremos envían un flujo de mensajes:

- *server streaming*, en el que el servidor devuelve una secuencia de respuestas a una única petición;
- *client streaming*, en el que el cliente envía una secuencia de mensajes y recibe una única respuesta; y
- *bidireccional*, en la que ambos envían secuencias de forma independiente.

Estos modos se declaran en la interfaz anteponiendo la palabra clave `stream` al tipo del parámetro, al del resultado o a ambos.

23.6. RMI

RMI traslada el mecanismo de invocación remota al modelo de objetos. La arquitectura es la misma que en RPC—interfaz, compilador, *stubs* y *runtime*—, pero el destino de una invocación no es un procedimiento del servidor, sino un método de un objeto concreto. Esa diferencia, aparentemente pequeña, tiene varias consecuencias.

En RPC, el servidor ofrece un conjunto plano de procedimientos, y toda invocación de una operación llega a la misma implementación. Si el servicio gestiona varias entidades independientes —varios ficheros, varias cuentas o varios contadores— el procedimiento no tiene forma de distinguirlas por sí mismo; el cliente tiene que incluir en cada invocación un argumento adicional que identifique la entidad, y el servidor tiene que mantener una tabla que asocie ese identificador con su estado. Eso es lo que hace NFS, por ejemplo, con los manejadores de fichero.

En RMI, en cambio, cada objeto remoto encapsula su propio estado, y el cliente invoca los métodos directamente sobre el objeto que le interesa. Un mismo servidor puede alojar muchos objetos que implementan la misma interfaz: dos contadores remotos llevan cuentas independientes aunque compartan servidor, tipo y código, igual que dos instancias de una misma clase en un programa convencional.

Para lograrlo, cada objeto remoto tiene una **referencia a objeto remoto**, que lo identifica de forma única en todo el sistema. De un modo u otro, esta referencia remota incluye el *endpoint* del servidor que lo aloja y una *identidad* que identifica el objeto. En el cliente, la referencia se materializa en un *proxy*, que es parte del *stub* de cliente y está ligado a un objeto concreto de ese servidor. La invocación, por tanto, requiere dos niveles de despachado² en el servidor. Primero un componente del *runtime*, habitualmente llamado *adaptador de objetos*, localiza el objeto a partir de su identidad. Después, el *stub* de servidor de ese objeto localiza el método a partir del identificador de operación. El objeto que implementa la interfaz en el servidor se suele denominar *sirviente* (*servant*).

La ventaja más importante de este modelo es que las referencias a objetos remotos son «ciudadanos de primera clase» (*first-class citizens*), es decir, valores que se pueden pasar como argumento o devolver como resultado de una invocación, igual que una referencia a un objeto convencional. Eso tiene varias consecuencias:

- El conjunto de objetos accesibles no está fijado de antemano. Un servidor puede crear objetos nuevos durante la ejecución y devolver sus referencias al cliente, *p. ej.* un objeto «factoría» cuyo método crea una sesión y devuelve la referencia al nuevo objeto.
- Los parámetros admiten dos modos de paso. Los objetos remotos se pasan por referencia: el receptor obtiene un proxy y sus invocaciones llegan al objeto original. El resto de valores (números, cadenas, es-

²también llamado «indirección»

estructuras) se pasan por valor: se serializan y el receptor obtiene una copia. En RPC solo existe el paso por valor.

- El cliente puede pasar al servidor la referencia de un objeto propio para que el servidor lo invoque más tarde, *p. ej.* para notificarle un evento. Esto es lo que llamamos *callback*, y es una de las situaciones en las que los roles de cliente y servidor de la invocación se invierten para una interacción concreta.

Por lo demás, las interfaces de RMI heredan las características del modelo de objetos del lenguaje: pueden extender otras interfaces (herencia de interfaz), y un proxy de una interfaz derivada se puede usar allí donde se espera una base (*upcasting*). Los errores del servidor se propagan como excepciones, que el *stub* de cliente vuelve a lanzar en el cliente.

Todo esto por supuesto tiene un coste. La vida de un objeto remoto debería terminar cuando ningún cliente lo necesita, pero el servidor no puede saber cuándo han dejado de existir los *proxies* que lo referencian en otros nodos. Algunos middlewares incorporan para eso un recolector de basura distribuido; otros dejan esa responsabilidad al programador, que tiene que destruir explícitamente los objetos que ya no necesita. Además, la facilidad para crear y repartir referencias invita a diseños con muchas invocaciones pequeñas, en las que la latencia de la red —que el modelo de objetos no oculta— acaba suponiendo una sobrecarga no trivial.

23.6.1. pico-rmi

Pico-rmi es la versión orientada a objetos de pico-rpc (§ 23.5.1). Mantiene la misma estructura —*runtime* (`picormi.py`), compilador (`rmigen.py`), definición de la interfaz en Python y serialización con `struct`— y solo cambia lo necesario para que un servidor pueda alojar varios objetos independientes.

La interfaz `Counter`, del ejemplo /pico-rmi, declara dos operaciones sin parámetros: `increment()` y `get()`. La mayor diferencia respecto a pico-rpc no está en la interfaz, sino en que ahora puede haber varios objetos que la implementan.

Cliente

El *stub* del cliente para un objeto remoto concreto se llama *proxy*, y para instanciarlo se requiere, además del *endpoint* del servidor, la identidad del objeto. Esta combinación es la «referencia de objeto remoto» o simplemente «referencia remota».

```

from counter_client_stub import CounterProxy

if len(sys.argv) != 3:
    print("usage: ./client <server> <identity>")
    exit(1)

server = sys.argv[1]
identity = sys.argv[2]

counter = CounterProxy(identity, server, 2001)
print("get() = {}".format(counter.get()))
print("increment() = {}".format(counter.increment()))
print("increment() = {}".format(counter.increment()))
print("get() = {}".format(counter.get()))

```

LISTADO 23.12: Cliente del ejemplo counter de pico-rmi
 [pico-rmi/client.py](#)

El mensaje de petición incluye en este caso la identidad del objeto (16 bytes) seguida del identificador de operación (1 byte), y los argumentos si los hubiera.

```
request = struct.pack('16sB', self.identity, self.INCREMENT)
```

LISTADO 23.13: Mensaje de petición pico-rmi generado en el *stub* del cliente

Servidor

En pico-rpc el *stub* de servidor es un módulo con funciones sueltas que reciben la implementación como argumento. En pico-rmi el compilador genera una clase (`Counter`) que hereda de `Servant`, con un método *stub* por cada operación y la misma tabla `STUBS`. El sirviente (`CounterI`) hereda de esa clase, así que cada *stub* invoca la operación sobre la propia instancia (`self`) y, por tanto, sobre su estado.

El *dispatcher* de pico-rpc se sustituye por el «adaptador de objetos» según la terminología habitual de RMI, que mantiene un diccionario de sirvientes indexado por la identidad del objeto (Listado 23.14). Al recibir una petición, obtiene la identidad y localiza el sirviente, que a su vez localiza el *stub* a partir del identificador de operación. Estos son los dos niveles de despachado descritos al principio de esta sección.

```

class Servant:
    STUBS: dict[int, Callable] = {}

    def dispatch(self, function_id, args):
        return self.STUBS[function_id](self, args)

class ObjectAdapter:
    def __init__(self, port):

```

```

self.port = port
self.servants = {}

def add(self, identity, servant):
    self.servants[identity] = servant

def dispatch(self, sock):
    request = sock.recv(MAX_MESSAGE_SIZE)
    identity = decode_identity(request[:IDENTITY_SIZE])
    function_id, args = request[IDENTITY_SIZE], request[IDENTITY_SIZE + 1:]

    servant = self.servants[identity]
    result = servant.dispatch(function_id, args)
    sock.sendall(result)

```

LISTADO 23.14: Sirviente base y adaptador de objetos de *runtime* de pico-rmi
 pico-rmi/picormi.py

El servidor crea dos sirvientes y los registra en el adaptador con las identidades `counter1` y `counter2` (Listado 23.15).

```

from picormi import ObjectAdapter
from counter_server_stub import Counter

class CounterI(Counter):
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
        return self.value

    def get(self):
        return self.value

adapter = ObjectAdapter(port=2001)
adapter.add('counter1', CounterI())
adapter.add('counter2', CounterI())
adapter.run()

```

LISTADO 23.15: Servidor del servicio Counter
 pico-rmi/server.py

Cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto cuya identidad recibe como argumento. Los dos contadores llevan cuentas independientes, aunque estén en el mismo servidor y compartan la misma interfaz. En pico-rpc eso habría requerido añadir un parámetro que identificara sobre qué contador debe aplicar la operación.

Pico-rmi aplica todas las simplificaciones de pico-rpc, y añade otras propias del modelo de objetos. Los sirvientes se registran al arrancar el servidor y no se pueden crear ni destruir después. Tampoco es posible pasar una referencia de objeto remoto como argumento o como valor de retorno, porque

Cliente

```
$ ./client.py 127.0.0.1 counter1
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 127.0.0.1 counter1
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 127.0.0.1 counter2
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

Servidor

```
$ ./server.py
Object adapter ready: ('0.0.0.0', 2001)
```

FIGURA 23.2: Cliente y servidor pico-rmi

solo admite enteros sin signo. Por último, si la identidad de la petición no corresponde a ningún sirviente, el adaptador no informa al cliente del error: la excepción termina el servidor.

23.6.2. Java RMI

Java RMI es el middleware de invocación remota que Sun incorporó a la plataforma Java en 1997. No dispone de un IDL ni de compilador de interfaces. La interfaz remota es una interfaz Java convencional, y el *stub* es un *proxy* dinámico que el *runtime* construye mediante reflexión. Eso simplifica mucho su uso, pero cliente y servidor estar escritos necesariamente en Java (o, al menos, deben ejecutarse en una JVM). Utiliza su propio protocolo llamado JRMP encapsulado sobre TCP y serializa los datos con la serialización nativa de Java. La semántica de invocación es *at-most-once*.

El ejemplo /java-rmi es el «hola mundo» de la documentación oficial de Java. Una interfaz remota (Listado 23.16) debe cumplir dos requisitos: extender *java.rmi.Remote*, una interfaz vacía que solo sirve para marcar el tipo como remoto, y declarar que todos sus métodos pueden lanzar *RemoteException*. Esta es una decisión de diseño deliberada; los fallos de comunicación no se deben ocultar al programador, por eso el compilador obliga al programador a tratarlo en cada invocación.

```
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}
```

LISTADO 23.16: Interfaz remota *Hello* en Java RMI

/java-rmi/Hello.java

El sirviente es una clase que implementa esa interfaz remota declarada (Listado 23.17). El método `exportObject()` de `UnicastRemoteObject` lo registra en el *runtime*, que queda escuchando en el puerto TCP indicado (5000), y devuelve el *stub* de cliente para ese objeto. El servidor Java RMI y el sirviente quedan definidos en este caso en una única clase `Server`.

El *runtime* mantiene activos sus hilos mientras haya objetos exportados, así que el proceso servidor no termina al salir de `main()`.

```
public class Server implements Hello {
    public String sayHello() {
        return "Hello, world!";
    }

    public static void main(String args[]) {
        try {
            Server obj = new Server();
            Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 5000);

            // Bind the remote object's stub in the registry
            Registry registry = LocateRegistry.getRegistry();
            registry.rebind("Hello", stub);

            System.err.println("Server ready");

        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.17: Servidor del ejemplo Hello en Java RMI
📄/java-rmi/Server.java

23.6.2.1. Registry

Para que el cliente pueda obtener ese *stub*, el servidor lo publica en el Registry con el nombre `Hello`. El Registry es un servicio de nombres muy sencillo que se ejecuta con el programa `rmiregistry`³ y escucha por defecto en el puerto 1099. El propio Registry es un objeto remoto, con identidad fija y conocida. Por eso, `LocateRegistry.getRegistry()` no necesita comunicarse con nada; construye localmente un *stub* a partir del nombre del nodo, el puerto 1099 y esa identidad fija. Así se resuelve el problema de obtener la primera referencia remota. Sin ella no se puede invocar nada.

³El Registry necesita acceder a la clase `Hello` para deserializar el *stub* que recibe, así que `rmiregistry` se debe lanzar desde el directorio que contiene las clases compiladas. Otra opción es que el propio servidor cree el Registry en su proceso con `LocateRegistry.createRegistry()`.

La invocación `registry.rebind("Hello", stub)` es a su vez una invocación remota sobre el Registry, en la que el *stub* viaja como argumento. Ese *stub* contiene la dirección IP y el puerto del servidor y la identidad del objeto. La dirección se obtiene resolviendo el nombre del nodo del servidor; si resuelve a una dirección de *loopback*, la referencia no sirve a los clientes de otros nodos. La propiedad `java.rmi.server.hostname` permite indicar la dirección correcta.

El cliente (Listado 23.18) obtiene una copia de ese *stub* con `lookup()`, y a partir de ese momento invoca `sayHello()` directamente sobre el servidor, en el puerto 5000; el Registry no interviene en las invocaciones.

```
public class Client {
    private Client() {}

    public static void main(String[] args) {
        String host = (args.length < 1) ? null : args[0];
        try {
            Registry registry = LocateRegistry.getRegistry(host);
            Hello stub = (Hello) registry.lookup("Hello");
            String response = stub.sayHello();
            System.out.println("response: " + response);
        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.18: Cliente del ejemplo Hello en Java RMI
 /java-rmi/Client.java

Por seguridad, el Registry solo acepta las operaciones de escritura —`bind()`, `rebind()` y `unbind()`— cuando proceden del mismo nodo. La consulta (`lookup`), en cambio, se puede invocar desde cualquier nodo.

23.6.2.2. Paso por valor y por referencia

Los objetos que implementan *Remote* y han sido exportados se pasan **por referencia**: al serializar un argumento o un resultado, el *runtime* los sustituye por su *stub*, como ocurre con el que el servidor entrega al Registry. El resto se pasan **por valor**, para lo que su clase debe implementar *java.io.Serializable*. Java RMI utiliza la serialización nativa de Java que, a diferencia de XDR o Protocol Buffers, no se limita a tipos declarados en una interfaz: serializa el grafo completo de objetos alcanzables desde el argumento, conservando su estructura.

23.6.2.3. Recolección de basura distribuida

Java RMI incorpora un recolector de basura distribuido que se basa en el conteo de referencias remotas. Cuando un proceso deserializa un *stub*, su *runtime* informa al servidor que aloja el objeto mediante una invocación `dirty()` sobre el DGC de ese servidor (otro objeto remoto con identidad fija), y cuando el recolector local elimina el último *proxy*, lo notifica con `clean()`.

Como el cliente puede terminar sin avisar, cada referencia se concede como un «préstamo» (*lease*) con una duración limitada, 10 minutos por defecto (propiedad `java.rmi.dgc.leaseValue`), que el cliente debe renovar periódicamente. Si el *lease* expira, el servidor decrementa el conteo. Cuando llega a cero, el *runtime* deja de mantener el objeto y el recolector local de la JVM del servidor lo elimina. Si el sirviente implementa la interfaz *Unreferenced*, se le notifica. Como cualquier esquema de conteo de referencias, no es capaz de recolectar ciclos de objetos que se referencian entre sí desde distintos nodos.

En el ejemplo, la única referencia local al sirviente es la variable `obj` de `main()`, que deja de existir cuando esta termina. El objeto sigue vivo porque el Registry mantiene una referencia remota —el *stub*— y renueva su *lease*.

23.6.2.4. Seguridad

JRMP no cifra ni autentica las comunicaciones. Para hacerlo, `exportObject()` admite una variante que recibe las factorías de *sockets* del cliente y del servidor; las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` permiten usar TLS.

Las primeras versiones de Java RMI permitían que el receptor descargara desde una URL indicada por el emisor (*codebase*) las clases que le faltaban. Las versiones actuales ya no lo permiten, así que ambos extremos necesitan tenerlas en su *classpath*.

Hay otro problema de seguridad más grave. La deserialización de objetos arbitrarios recibidos de la red es un vector de ataque para introducir código malicioso. Un atacante puede enviar un grafo de objetos, construido con clases presentes en el *classpath* del servidor, cuya deserialización acaba ejecutando código arbitrario. Desde Java 9, la plataforma incluye filtros de deserialización que limitan qué clases se aceptan. Aun así, exponer un servicio Java RMI en una red no confiable se considera una mala práctica.

23.6.3. ZeroC Ice

ICE es un middleware RMI desarrollado por la empresa ZeroC, cuya primera versión apareció en 2003. Sus autores procedían del mundo de CORBA y lo concibieron como un sustituto que conservara el modelo de objetos distribuidos, pero con una especificación mas sencilla y un *runtime* más ligero. Genera *stubs* para muchos lenguajes de modo que, igual que en gRPC, cliente y servidor pueden estar escritos en lenguajes distintos. Utiliza un protocolo de aplicación propio llamado ICEP, independiente del transporte. Sus mensajes se encapsulan sobre el que indique el *endpoint*.

Las piezas son las habituales de cualquier middleware RMI (§ 23.6): un IDL, un compilador que genera los *stubs* de cliente y servidor, un *runtime* con su adaptador de objetos y una referencia de objeto remoto que el cliente materializa en un proxy.

El ejemplo `🐍/py/counter-by-id` es el servicio Counter de pico-rmi (§ 23.6.1) reescrito para ICE: un servidor aloja dos contadores independientes y un cliente invoca sus operaciones.

La interfaz se describe en el lenguaje SLICE —el IDL de ICE— (Listado 23.19). Permite declarar módulos, interfaces, estructuras, secuencias, diccionarios, enumerados y excepciones; las interfaces admiten herencia, incluso múltiple.

```
    idempotent long get();
};
};
```

LISTADO 23.19: Interfaz *Counter* en SLICE
🐍/py/counter-by-id/counter.ice

Una peculiaridad de SLICE es que es posible declarar qué operaciones son idempotentes (*idempotent*). Sabiendo esto, el *runtime* del cliente puede decidir cómo gestionar un fallo. Una operación marcada así se puede reintentar sin romper la semántica *at-most-once* (§ ??), algo que no se puede aplicar a un método no idempotente. En Counter, *get()* es idempotente e *increment()* no lo es.

Cliente y servidor conocen esa marca porque sus *stubs* se han generado desde la misma interfaz, así que ninguno de los dos necesita que se la indiquen. Aún así, el «modo» de la operación viaja en cada petición. Está ahí para componentes que no tienen la interfaz, *p. ej.* un intermediario que reenvía invocaciones desconocidas, como el servicio de propagación de eventos IceStorm.

El compilador `slice2py` (`zeroc-ice-compilers`) genera el módulo `counter_ice.py` y un paquete `Example` con el proxy de cliente (`CounterPrx`) y el *stub* de servidor (`Counter`). Pero como Python es un lenguaje dinámico, ese paso se puede evitar. La función `Ice.loadSlice()` compila el fichero `.ice` al vuelo durante la ejecución y construye los tipos en memoria.

El sirviente (`CounterI`) hereda del *stub* de servidor e implementa las dos operaciones (Listado 23.20). Cada método recibe un parámetro que no aparece en la interfaz, `current`, que permite al sirviente acceder a los datos de la propia invocación: la identidad del objeto invocado, el nombre de la operación, el adaptador, o la conexión entre otros. Con ese parámetro, un único sirviente puede encargarse de muchos objetos distintos y distinguirlos por su identidad. Eso es lo que se llama «sirviente por defecto» (*default servant*), y evita crear un sirviente por cada entidad cuando son miles o millones.

```
class CounterI(Example.Counter):
    def __init__(self):
        self.value = 0

    def increment(self, current):
        self.value += 1
        return self.value

    def get(self, current):
        return self.value

def main(ic):
    adapter = ic.createObjectAdapter('CounterAdapter')

    proxy1 = adapter.add(CounterI(), Ice.stringToIdentity('counter1'))
    proxy2 = adapter.add(CounterI(), Ice.stringToIdentity('counter2'))

    print(proxy1)
    print(proxy2)

    adapter.activate()
    ic.waitForShutdown()
    return 0
```

LISTADO 23.20: Servidor del servicio Counter en ICE
 `/py/counter-by-id/server.py`

El servidor registra cada sirviente en el adaptador de objetos con `adapter.add()` bajo una identidad única, que se puede construir a partir de una cadena con `Ice.stringToIdentity()`. Ahí están los dos niveles de despachado: el adaptador localiza el sirviente a partir de la identidad que viene en la petición, y el *stub* del servidor localiza el método a partir del identificador de la operación. Es exactamente el mismo papel del `ObjectAdapter` de

pico-rmi, pero el *endpoint* en el que escucha no se le pasa como argumento, sino que lo toma de una propiedad de configuración:

```
CounterAdapter.Endpoints=tcp -h 127.0.0.1 -p 2002
```

Prácticamente todo el comportamiento del *runtime* se ajusta con propiedades como esta, que se leen del fichero indicado en `--Ice.Config` o bien de la línea de comandos. El protocolo de transporte por defecto, el tamaño máximo de mensaje, los plazos de espera, la cantidad de reintentos, el tamaño del *pool* de hilos, las trazas de red y muchas otras características son configurables con propiedades (hay cientos de ellas). El código no cambia al pasar de TCP a TLS, ni al mover el servidor a otro puerto, solo hay que ajustar las propiedades.

23.6.3.1. Referencias remotas textuales

El método `adapter.add()` devuelve el proxy del objeto recién registrado, e ICE le da una representación textual completa, lo que su documentación llama *stringified proxy*. Eso es precisamente lo que imprime el servidor del ejemplo:

```
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Esa cadena es una referencia de objeto remoto (§ 23.6) representada como texto, y contiene los dos elementos que la definen: la identidad del objeto (`counter1`), y el *endpoint* del servidor que lo aloja (`tcp -h 127.0.0.1 -p 2002`), después. Entre ambos hay varias opciones de la referencia: `-t` es el modo de invocación (*twoway*), `-e 1.1` la versión del formato de serialización y `-t 60000` el *timeout* de invocación en milisegundos.

En realidad cuando se utiliza `print()` con un proxy, se está invocando `proxyToString()`. Y también existe la función inversa `stringToProxy()`, que construye un proxy que el cliente puede invocar.

Que la referencia remota se pueda transportar como texto resuelve el problema de «la primera referencia», el mismo que Java RMI resuelve con el Registry (§ 23.6.2): el cliente la obtiene por cualquier medio —un fichero de configuración, un argumento en la línea de comandos, un mensaje de correo— sin necesidad de un servicio de nombres. También hace evidente algo que otros middlewares ocultan: una referencia remota puede llevar varios *endpoints*, incluso con protocolos distintos (`tcp`, `ssl`, `udp`), y el *runtime* prueba cada uno de ellos hasta conseguir establecer la comunicación. Si el adaptador del ejemplo no fijara la dirección con `-h`, escucharía en todas las del nodo y la referencia incluiría un *endpoint* por cada una.

23.6.3.2. Reflexión remota

El método `stringToProxy()` solo considera la cadena que se le pasa (el proxy textual); no comprueba que el objeto exista ni que sea del tipo esperado. Devuelve un proxy de tipo `Ice::ObjectPrx`, la clase base de todos los proxies. Para invocar un método de la interfaz declarada —`increment()` en este ejemplo— es necesario que el tipo de este proxy sea del tipo específico (`CounterPrx`). De eso se encarga `checkedCast` (Listado 23.21).

```
def main(ic):
    proxy = ic.stringToProxy(sys.argv[1])
    counter = Example.CounterPrx.checkedCast(proxy)

    if counter is None:
        raise RuntimeError('invalid proxy')

    print("get() = {}".format(counter.get()))
    print("increment() = {}".format(counter.increment()))
    print("increment() = {}".format(counter.increment()))
    print("get() = {}".format(counter.get()))
```

LISTADO 23.21: Cliente del servicio Counter en ICE
 `/py/counter-by-id/client.py`

Convertir una referencia de un tipo base a uno derivado es un **molde hacia abajo** (*downcasting*), una operación habitual en cualquier lenguaje orientado a objetos y que no es nada particular de ICE o RMI. Lo que sí cambia es quién puede comprobarlo. En local, el objeto está en memoria y el lenguaje puede hacerlo, pero con RMI el único que sabe de qué tipo es el objeto es el servidor. Por eso, `checkedCast()` en realidad implica una invocación remota que interroga al objeto para verificar su tipo. Hay una variante `uncheckedCast()` que construye el proxy, pero no comprueba, algo que lógicamente implica un riesgo de fallo, pero hay situaciones en las que simplemente no es posible hacer la comprobación.

Preguntarle a un objeto por su propio tipo es un mecanismo de **reflexión**, «remota» en este caso. Toda interfaz `SLICE` hereda implícitamente de `Ice::Object`, que aporta a cualquier objeto remoto las operaciones necesarias para preguntarle sobre sí mismo. Su alcance, eso sí, es corto: el objeto revela qué interfaces implementa (método `ice_ids`), pero no qué operaciones tiene cada una ni cómo se serializan sus argumentos. ICE no ofrece un repositorio de interfaces que se pueda consultar en ejecución, como sí tenía CORBA.

En el ejemplo `counter`, cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto de la referencia recibida co-

mo argumento. Igual que en pico-rmi, los dos contadores llevan cuentas independientes aunque compartan servidor, adaptador e interfaz.

Servidor

```
$ ./server.py --Ice.Config=server.config
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
counter2 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Cliente

```
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 'counter2 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

FIGURA 23.3: Servidor y cliente ICE
 /py/counter-by-id

23.6.3.3. Paso de referencias y ciclo de vida

En SLICE, el tipo de un proxy se escribe con un asterisco tras el nombre de la interfaz (Counter*), y se puede usar como parámetro o como resultado de una operación. Las referencias de objeto remoto son, por tanto, «ciudadanos de primera clase» (§ 23.6), con los dos modos de paso que eso implica. Los objetos remotos se envían por referencia, y el receptor construye con ella un proxy cuyas invocaciones llegan al original; el resto de los tipos —números, cadenas, estructuras, secuencias, diccionarios— se envían por valor. La diferencia con Java RMI, que serializa el grafo completo de objetos alcanzables (§ 23.6.2), es que aquí solo se envía lo que declare la interfaz.

Donde ICE se aparta de Java RMI es en el ciclo de vida. ICE no incorpora recolector de basura distribuido. Es responsabilidad del programador. A cambio, el servidor no mantiene conteos de referencias ni (*leases*) que renovar, y un cliente que desaparece sin avisar no deja nada pendiente.

23.6.3.4. Modos de invocación

El modo de invocación no forma parte de la interfaz; es un atributo de la referencia remota, que el cliente puede cambiar en su proxy cuando necesite.

Corresponde con el `-t` que aparece en el proxy textual, aunque el objeto `Ice::ObjectPrx` también tiene métodos para lograr lo mismo en ejecución. El modo determina el estilo de intercambio y, por tanto, la semántica de invocación que obtiene el cliente (Cuadro 23.1).

Opción	Modo	Intercambio	Semántica	Método <code>ObjectPrx</code>
<code>-t</code>	<i>twoway</i>	RR sobre TCP	<i>at-most-once</i>	<code>ice_twoway()</code>
<code>-o</code>	<i>oneway</i>	R sobre TCP	<i>maybe</i>	<code>ice_oneway()</code>
<code>-d</code>	<i>datagram</i>	R sobre UDP	<i>maybe</i>	<code>ice_datagram()</code>
<code>-O</code>	<i>batch oneway</i>	R sobre TCP	<i>maybe</i>	<code>ice_batchOneway()</code>
<code>-D</code>	<i>batch datagram</i>	R sobre UDP	<i>maybe</i>	<code>ice_batchDatagram()</code>

CUADRO 23.1: Modos de invocación en una referencia remota ICE

El modo *twoway* (por defecto) es el único que permite obtener respuestas. El cliente recibe el resultado o una excepción, y como TCP evita pérdidas y duplicados, y el *runtime* no reintenta por su cuenta una operación que no sea idempotente, la semántica es *at-most-once* (§ 23.4).

Los otros cuatro cubren solo el mensaje de petición. El *stub* de cliente lo entrega al transporte y devuelve el control inmediatamente, sin esperar. El cliente no puede saber si la operación llegó a ejecutarse. Incluso con TCP, que garantiza que el mensaje llega al extremo remoto, no es posible saber si el sirviente ejecutó correctamente la operación (fallo parcial). Por eso la semántica es *maybe* en los cuatro casos. Lo que los distingue es el coste y el riesgo: con `-d` y `-D` el mensaje puede perderse por el camino, mientras que las variantes «por lotes» (*batch*) no envían nada hasta que se vacía la cola, lo que amortiza la sobrecarga cuando las invocaciones son muchas y pequeñas, a cambio de perderlas todas juntas si hay un problema en la conexión.

Por coherencia, un proxy de solo petición únicamente admite operaciones sin resultado ni excepciones declaradas; invocar `get()` en esos casos produce un error local (`TwowayOnlyException`) antes de enviar nada. La misma interfaz sirve para los dos estilos de intercambio (§ 23.1) sin cambiar la definición, algo que por ejemplo en gRPC exigiría declararlo en el fichero `.proto`.

Existe una opción más en los proxies textuales: `-s`, que no es un modo sino un requisito: obliga al *runtime* a usar únicamente *endpoints* seguros, y por eso aparece combinada con la del modo (`-t -s`).

23.7. Localización de servicios

De uno u otro modo, cualquier referencia de objeto remoto incluye el *endpoint* del servidor que aloja al objeto (§23.6). Eso resuelve el problema de encontrarlo, pero crea otro: acopla el cliente a un c:oncreto. Así lo hacen todos los ejemplos de este capítulo: pico-rmi recibe el host como argumento, gRPC construye su canal con un *endpoint* fijo, el proxy de ICE lo lleva incluido, etc. Esto funciona mientras nada cambie, pero en un despliegue las cosas cambian. Los servidores cambian de nodo (se «migran»), se replican, o se reinician en otro puerto. También es útil poder arrancar los servidores solo cuando hacen falta («bajo demanda»).

La **transparencia de localización** ofrece a los clientes nombres alternativos para los servidores que no incluyen endpoints. Para lograr la comunicación obviamente los endpoints son necesarios, pero se puede aplazar el momento de averiguarlos hasta que se produzca la invocación. Para lograr esto, el middleware debe incluir un servicio que proporcione los endpoints —potencialmente efímeros— para los identificadores estables de los servicios. El componente que hace esto es el *binder*.

Todo *binder* tiene el problema del *bootstrapping*, es decir, ¿cómo se obtienen sus endpoints, si para eso hace falta precisamente el servicio que ofrece el *binder*? Es el mismo problema de los servidores DNS, y la solución también suele ser la misma. El *binder* (al igual que el servidor DNS) tiene un *endpoint* fijo, conocido y estable, ya sea por convención o por configuración. El *binder* de ONC-RPC es *rpcbind* (§23.5.2), que escucha siempre en el puerto 111, y el de Java RMI es el Registry (§23.6.2), en el 1099 y con una identidad fija. En ICE ese papel lo realiza un servicio llamado *Locator*, cuyo proxy se fija con la propiedad `Ice.Default.Locator`, que tiene este aspecto:

```
Ice.Default.Locator=IceGrid/Locator:tcp -h maine -p 4061
```

La alternativa al *binder* es el «descubrimiento de servicios». En lugar de preguntarle a un servicio conocido, se envía una consulta a la red con algún mecanismo de difusión —*rpcbind* e ICE lo tienen. El problema es que la difusión normalmente está localizada únicamente a la LAN o la subred.

23.7.1. Registro de servidores y objetos

Para que el *binder* pueda hacer su trabajo, los servidores tienen registrarse en él. En ONC-RPC el servidor registra en *rpcbind* cada combinación de programa, versión y protocolo junto al puerto que el sistema le ha asignado. La tabla resultante se consulta con *rpcinfo -p*. Con el servidor del ejemplo /onc-rpc-hello en marcha, sus dos últimas entradas corresponden al

programa 600000000 que declara su interfaz, registrado por partida doble porque acepta los dos transportes:

```
$ rpcinfo -p
  program vers proto  port  service
  100000    4   tcp    111  portmapper
  100000    3   tcp    111  portmapper
  100000    2   tcp    111  portmapper
  100000    4   udp    111  portmapper
  100000    3   udp    111  portmapper
  100000    2   udp    111  portmapper
  600000000 1   udp   57269
  600000000 1   tcp   62357
```

Los puertos son aleatorios y cambian en cada arranque; eso es justamente por lo que hace falta un *binder*, y lo que permite por ejemplo que un servidor NFS no necesite un puerto fijo. Obsérva que *rpcbind* figura en su propia tabla (100000 «portmapper»). El binder es un programa RPC más, igual que el Registry de Java RMI es un objeto remoto Java y el localizador de ICE un objeto ICE.

En Java RMI el registro es explícito y lo hace el programador con *rebind()*, que publica el *stub* bajo un nombre. El Registry *stub* completo, no solo el endpoint.

ICE lo automatiza por completo. El adaptador de objetos, al activarse, registra por su cuenta en el *binder* el «proxy directo» (*direct proxy*) —el que lleva *endpoints*, como los de § 23.6.3— en el que va a escuchar; el programador no tiene que escribir nada de código, solo le da al adaptador un identificador y configura un Locator, ambos por configuración (Listado 23.22). Obsérva que el adaptador ni siquiera fija el puerto (*tcp*), porque no hace falta: el *runtime* elige uno libre y es el *binder* quien lo almacena.

```
PrinterAdapter.Endpoints=tcp
PrinterAdapter.AdapterId=PrinterAdapter1
PrinterAdapter.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

LISTADO 23.22: Configuración del servidor con localizador
📄/locator/server.config

El servidor es exactamente el mismo del «hola mundo» (📄/py/hello/server.py). Lo único que cambia es lo que imprime: el proxy ya no lleva *endpoints*, sino el nombre del adaptador tras una @. Es un «proxy indirecto» (*indirect proxy*).

```
$ ./server.py --Ice.Config=../locator/server.config
printer1 -t -e 1.1 @ PrinterAdapter1
```

23.7.2. Resolución de nombres

La resolución es transparente para el programador. En Java RMI la consulta al Registry es una línea de código que alguien tiene que escribir (`lookup()`), y ocurre una vez, antes de tener el proxy. En ONC-RPC, `clnt_create()` hace lo propio contra `rpcbind`. En ICE no hay ninguna llamada equivalente; el cliente construye el proxy y lo invoca, y es el *runtime* el que, al intentar establecer la comunicación, observa que la referencia no tiene *endpoints* y consulta al localizador antes de conectar. El cliente solo necesita saber dónde está el localizador (Listado 23.23) y usar como referencia la que imprimió el servidor. Los *endpoints* que el *runtime* obtiene quedan en una caché para evitar resoluciones adicionales innecesarias.

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

LISTADO 23.23: Configuración del cliente con localizador
 locator/locator.config

```
$ ./client.py --Ice.Config=locator.config "printer1 -t -e 1.1 @ PrinterAdapter1"
```

Esa caché caduca por fallo, no por tiempo. Cuando el *runtime* no consigue conectar con los *endpoints* cacheados, descarta la entrada, vuelve a preguntarle al localizador y reintenta la invocación. Esa *re-resolución* es lo que da transparencia de verdad. Permite mover un servidor de puerto o de nodo mientras un cliente lo está usando, sin que reciba ningún error. Aunque solo oculta los fallos de conexión; si la petición ya salió y la operación no es *idempotent*, reintentar rompería la semántica *at-most-once* y el error sí llega al cliente.

La diferencia con los otros dos binders es evidente. El *stub* que devuelve el Registry de Java RMI lleva el *endpoint* y no se renueva nunca. Si el servidor cambia de puerto, ese *stub* queda inservible y el programador es quien tiene que repetir la llamada a `lookup()`. Lo mismo ocurre con `clnt_create()` en ONC-RPC, que consulta a `rpcbind` al crear el manejador y no vuelve a hacerlo. En ambos casos el *binder* resuelve el arranque; en ICE, además, vigila la validez de la referencia durante toda su vida. Cap'n Proto adopta un enfoque distinto, que renuncia al binder por diseño hasta el punto de considerarlo una importante debilidad (§23.9).

23.8. Invocación y despacho asíncrono

Las invocaciones que se han tratado hasta ahora son «síncronas», es decir, el hilo del cliente queda bloqueado desde que envía la petición hasta que llega la respuesta. Se hace así como analogía a la forma en que funcionan

las llamadas locales convencionales. Sin embargo, en una invocación remota ese tiempo de espera —RTT más lo que tarde el servidor en ejecutar la operación— se pierde. Hay dos mecanismos para aprovecharlo, uno en cada extremo, y son independientes porque resuelven esperas distintas. La *invocación asíncrona*, de AMI (Asynchronous Method Invocation), permite al cliente seguir ejecutando código en lugar de esperar, y el *despachado asíncrono*, AMD (Asynchronous Method Dispatching), evita que se bloquee el servidor. Ninguno de los extremos puede saber cómo lo está haciendo el otro, porque el intercambio de mensajes es el mismo RR de siempre. Una invocación asíncrona sigue esperando una respuesta, la diferencia es que el programa no se detiene a esperarla. No se debe confundir la invocación asíncrona con la invocación de solo petición, que no tiene respuesta.

Los ejemplos de esta sección utilizan ICE, que es el middleware más completo de los tratados en este aspecto.

23.8.1. Invocación asíncrona

En ICE la decisión es del cliente y no afecta a la interfaz. Para la operación `factorial()` de la interfaz *Math*, declarada como `long factorial(int value)`, el compilador genera siempre dos formas de invocarla en el proxy: `factorial()`, que bloquea, y `factorialAsync()`, que devuelve inmediatamente un *future*. Un *future* representa un resultado que todavía no está disponible.

El programa puede recoger el resultado de varias formas:

- Esperando más tarde: el cliente sigue con su trabajo y bloquea solo cuando necesita el valor. La llamada `result()` espera si la respuesta aún no ha llegado (Listado 23.24).
- Con un *callback*: el cliente registra una función que el *runtime* invoca cuando el *future* se completa, y no espera en ningún momento (Listado 23.25).
- En un bucle de eventos: `Ice.wrap_future()` convierte el *future* en uno de *asyncio*, de modo que se puede usar `await` (Listado 23.26).
- Consultando periódicamente si ya ha llegado el resultado (*polling*), con `done()` (Listado 23.27).

```
future = math.factorialAsync(value)
print("That was an async call")
print(f"Async result is: {future.result()}")
```

LISTADO 23.24: Cliente que espera el resultado más tarde

 /py/ami/client-block.py

```
def callback(future):
    try:
        print(f"Callback: value is: {future.result()}")
    except Exception as ex:
        print(f"Exception is: {ex}")

def main(ic):
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} <proxy> <value>")
        return 1

    proxy = ic.stringToProxy(sys.argv[1])
    math = Example.MathPrx.checkedCast(proxy)
    value = int(sys.argv[2])

    if not math:
        raise RuntimeError('Invalid proxy')

    future = math.factorialAsync(value)
    future.add_done_callback(callback)

    print("That was an async call")

    return 0
```

LISTADO 23.25: Cliente con *callback* /py/ami/client-callback.py

```
result = await Ice.wrap_future(math.factorialAsync(value))
print(f"Async result is: {result}")
```

LISTADO 23.26: Cliente integrado en un bucle de eventos

 /py/ami/client-await.py

```
future = math.factorialAsync(value)
while not future.done():
    print("Still waiting...")
    time.sleep(0.1)
print(f"Async result is: {future.result()}")
```

LISTADO 23.27: Cliente que consulta periódicamente

 /py/ami/client-poll.py

Que la invocación no bloquee tiene consecuencias. La excepción no se puede lanzar en el punto de la llamada, que ya ha terminado, sino que incluye en el *future* y se eleva al pedir el resultado; por eso el *callback* del ejemplo captura la excepción al leerlo. Además, el *callback* normalmente se ejecuta en un hilo del *runtime*, no en el que hizo la invocación, así que si comparte datos con el resto del programa hay que sincronizar el acceso. Y la semántica de invocación no cambia; una invocación asíncrona sigue siendo *at-most-once*,

igual que la síncrona. Cambia cuándo la recibe el cliente, no qué garantías tiene.

23.8.2. Despachado asíncrono

En el servidor el problema es otro. El *runtime* atiende las peticiones con un *pool* de hilos limitado —por omisión, de un solo hilo, ajustable con la propiedad `Ice.ThreadPool.Server.Size`—, y el hilo que recibe una petición ejecuta el sirviente y bloquea hasta que retorna. Si la operación es lenta, los hilos del *pool* quedan ocupados. Unas pocas peticiones simultáneas pueden agotar el *pool* e impedir que se atiendan las demás. El despachado asíncrono desacopla la ejecución de la operación del envío de la respuesta. Con AMD el sirviente devuelve el control al *runtime* sin tener el resultado, y este envía la respuesta más tarde, cuando el resultado esté disponible.

A diferencia de la invocación asíncrona, esta decisión sí aparece en la interfaz, porque cambia la firma que tiene el sirviente. En el SLICE se declara con una anotación en la operación, `[.amd] long factorial(int value)`. El sirviente del ejemplo (Listado 23.28) no calcula nada; crea un *future*, lo entrega con el argumento a una cola de trabajo que atiende otro hilo, y devuelve un *future*. La respuesta no sale al retornar el método, sino cuando se complete ese *future*. En el ejemplo lo hace el *job* al ejecutarse (Listado 23.29), con `set_result()`, o con `set_exception()` si tiene que cancelarse, para que el cliente reciba un error en lugar de esperar indefinidamente.

```
class MathI(Example.Math):
    def __init__(self, work_queue):
        self.work_queue = work_queue

    def factorial(self, value, current=None):
        future = Ice.Future()
        self.work_queue.add(future, value)
        return future
```

LISTADO 23.28: Sirviente con despachado asíncrono
 `/py/amd/server.py`

```
class Job:
    def __init__(self, future, value):
        self.future = future
        self.value = value

    def execute(self):
        self.future.set_result(factorial(self.value))

    def cancel(self):
        self.future.set_exception(Example.RequestCanceledException())
```

LISTADO 23.29: Tarea (*job*) que completa el *future*

/py/amd/work_queue.py

El cliente que se utiliza con este servidor AMD es uno síncrono corriente, que invoca `factorial()` y espera el resultado. Este mecanismo no hace que el cálculo sea más rápido. El factorial es cálculo puro y la cola de trabajo no le aporta nada. La ventaja aparece cuando la operación espera, *p. ej.* una consulta a una base de datos o a otro servicio remoto, porque mientras espera no bloquea ningún hilo del *runtime*.

23.9. Cap'n Proto

La arquitectura de Cap'n Proto es la de cualquier RMI (§ 23.6): un IDL, un compilador, *stubs* y un *runtime*, con un modelo de objetos distribuidos. Sin embargo, cuestiona varias decisiones de diseño que se dan por hecho en la mayoría de los otros middlewares: la necesidad de serializar los datos, el modelo de referencias remotas y la forma de obtener las referencias a los objetos.

Las siguientes secciones examinan cada una de ellas sobre el mismo ejemplo (/capn-rpc-hello), un servicio *TextProcessor* con una única operación, `upper()`, que devuelve una versión en mayúsculas del texto recibido. Existe en C++ y en Python, con el mismo fichero de interfaz (Listado 23.30). El fichero comienza con un identificador de 64 bits que lo identifica de forma inequívoca, y cada método lleva un número de orden (`@@`) que cumple el mismo papel que el número de campo de Protocol Buffers, es decir, permite ampliar la interfaz con métodos nuevos sin romper la compatibilidad.

```
@0x8b96e1f756a41043;

interface TextProcessor {
  upper @@ (message :Text) -> (result :Text);
}
```

LISTADO 23.30: Interfaz *TextProcessor* en Cap'n Proto

/capn-rpc-hello/py/upper.capnp

El compilador `capnp` ( `capnp`) genera el código de los *stubs* para C++. En Python no hace falta ese paso: el módulo `capnp` carga el fichero de interfaz directamente al importar `upper_capnp`, de un modo similar a `Ice.loadSlice()`.

23.9.1. ¿Sin serialización?

Enviar un argumento con XDR, Protocol Buffers o la serialización de Java implica convertirlo desde su representación en memoria a una secuencia de bytes, y el receptor debe hacer la conversión inversa. Cap'n Proto elimina

ese paso: el mensaje se construye directamente en un *buffer* con la misma disposición que tendrá en el cable, y el receptor lo lee sin transformarlo. Esto es lo que se llama serialización *zero-copy*.

Para que esto sea viable, los tipos que genera el compilador no son objetos cuyos campos se rellenan al deserializar, sino accesoros que operan sobre el *buffer*. En el cliente del ejemplo (Listado 23.32), el argumento `message` de `upper()` escribe el texto directamente en el mensaje que se va a enviar, y el resultado (`reply.result`) se lee del *buffer* tal como llegó.

Un mensaje que no necesita interpretarse para leerlo se puede leer en parte sin recorrer el resto, procesar antes de haberlo recibido completo, o guardar en un fichero y proyectar en memoria sin releerlo. Como contrapartida, ocupa más que uno con codificación compacta.

23.9.2. Referencias como *capabilities*

En Cap'n Proto las referencias a objetos remotos se llaman *capacidades* (*capabilities*). Son la referencia de objeto remoto de § 23.6, pero con una propiedad añadida: designan un objeto y, además, le dan el permiso para invocarlo. En Java RMI y en ICE una referencia es un dato, una identidad más un *endpoint*, y con eso basta para poder invocar el objeto. En Cap'n Proto una capacidad no se puede fabricar ni se puede delegar a un tercero. Cuando un nodo envía una referencia a otro, el identificador únicamente es válido para esa conexión. Nadie que no lo haya recibido directamente puede invocar.

Colocar la referencia y el permiso juntos no es un mero detalle de implementación; es la respuesta a un problema de seguridad. Cuando una referencia se puede conseguir sin que nadie la haya concedido, el permiso hay que comprobarlo en otro lugar. El servidor debe averiguar quién es el cliente e implementar un mecanismo de autorización.

Ese enfoque conlleva dos inconvenientes:

- Cada cliente tiene permiso para acceder a todo aquello que pueda nombrar, y no solo de lo que necesita para su tarea. Viola el principio de mínimo privilegio.
- El problema del «diputado confundido» (*confused deputy*): un servicio con autoridad que actúa en nombre de un cliente, sobre un recurso cuyo nombre le indica el cliente, puede acabar usando esa autoridad sobre un recurso distinto para el que el cliente no tiene permiso.

Con capacidades, el cliente no le indica al servicio un nombre, sino que le entrega la referencia, que designa el recurso y prueba a la vez que tiene permiso para usarlo. La política de acceso deja de ser una tabla que mantener y pasa a ser una decisión de diseño de las interfaces: para que alguien solo pueda leer un fichero, se le entrega un objeto que solo ofrece lectura, y no hay nada más a lo que pueda llegar.

Esta idea también cambia la forma de obtener la primera referencia. No hay Registry, servicio de nombres ni referencia con identidad conocida. Al establecer la conexión, cada extremo puede ofrecer un único objeto inicial, el *bootstrap*, que el otro obtiene como primera capacidad. En el servidor del ejemplo (Listado 23.31), es el argumento `bootstrap` de `TwoPartyServer`, y el cliente lo recoge con el método `bootstrap()` (Listado 23.32). La conexión es a la vez el canal y el nombre.

```
class TextProcessorI(upper_capnp.TextProcessor.Server):
    async def upper(self, message, _context):
        print("Client sent:", message)
        return message.upper()

    async def new_connection(stream):
        await capnp.TwoPartyServer(stream, bootstrap=TextProcessorI()).on_disconnect()
```

LISTADO 23.31: Servidor de *TextProcessor* en Cap'n Proto
[📄/capn-rpc-hello/py/server.py](#)

```
async def main(host, port):
    connection = await capnp.AsyncIOStream.create_connection(
        host=host, port=port)

    client = capnp.TwoPartyClient(connection)
    text_processor = client.bootstrap().cast_as(upper_capnp.TextProcessor)

    reply = await text_processor.upper(message="hello, world!")
    print("Server reply:", reply.result)
```

LISTADO 23.32: Cliente de *TextProcessor* en Cap'n Proto
[📄/capn-rpc-hello/py/client.py](#)

La ausencia de un servicio de nombres es deliberada, no una carencia: un servicio de nombres es justamente una fuente de referencias que se consiguen sin que nadie las haya concedido, como el `lookup()` de Java RMI o la referencia textual de ICE. Los objetos se alcanzan navegando desde el objeto inicial, que entrega a cada cliente solo lo que debe ver; si una aplicación necesita algo parecido a un servicio de nombres, lo implementa como un objeto más. Esto no resuelve el problema de localización de § ?? : el cliente sigue necesitando el *endpoint* del servidor al que se conecta.

Las capacidades están ligadas a la conexión, lo que simplifica el ciclo de vida sin necesidad de *leases*: un objeto se cierra cuando se pierden todas sus referencias o la conexión por la que se recibieron. En el ejemplo, el servidor crea un sirviente por conexión (`new_connection()` en el listado), que desaparece con ella y obliga al cliente a reconectar. El protocolo se transporta sobre un flujo de bytes, como TCP en el ejemplo.

Este modelo tiene contrapartidas. Revocar una capacidad ya entregada es difícil, y responder a quién tiene acceso a un objeto exige reconstruir el grafo de referencias repartidas, sin una tabla central que consultar. Una caída de conexión invalida todas sus capacidades, y el cliente debe reconectar y reconstruir los objetos que usaba. Guardar una capacidad para más tarde requiere el segundo nivel del protocolo (§ 23.9.4), y no todas se pueden guardar. Por último, sin servicio de nombres el descubrimiento queda fuera del middleware.

23.9.3. Promesas y *pipelining*

Cap'n Proto no intenta que una invocación remota se parezca a una local. Toda invocación devuelve inmediatamente una promesa (*promise*)⁴, un resultado que todavía no está disponible. El cliente decide con `await` cuándo esperarlo. El diseño de Cap'n Proto pone mucho incapié en no ocultar la red y sus limitaciones, y para lograrlo, la promesa es esencial para aplicar optimizaciones.

Si las referencias son ciudadanos de primera clase, un patrón habitual es invocar una operación que devuelve un objeto e invocar de inmediato otra sobre ese resultado. Con un servicio de ficheros, cuyo método `open()` devuelve un objeto *Directory* o *File*, la lectura de un fichero requiere tres invocaciones encadenadas, aquí en pseudocódigo:

```
directory = root.open("bar")
file = directory.open("foo")
data = file.read()
```

En cualquier otro middleware, cada una tiene que esperar a que llegue la respuesta de la anterior para poder usar su resultado, lo que cuesta tres viajes de ida y vuelta. En Cap'n Proto, la promesa que devuelve `open()` ya ofrece los métodos del objeto que devolverá, y las invocaciones sobre ella se envían al servidor sin esperar: el cliente cursa las tres peticiones juntas, cada una con una referencia al resultado de la anterior, y el servidor las va ejecutando en cadena. Es lo que se llama *promise pipelining*, y reduce la secuencia a un único viaje de ida y vuelta. Su documentación lo presenta

⁴El mismo concepto que el *future* de § 23.8

como un «viaje en el tiempo»: el cliente recibe la promesa, y puede usarla como destino de nuevas invocaciones, antes de que el servidor haya recibido siquiera la primera petición. Solo funciona cuando todos los objetos de la cadena los aloja el mismo servidor.

Esto ataca directamente el coste que § 23.6 atribuía al modelo de objetos: la facilidad para repartir referencias invita a interfaces con muchas invocaciones pequeñas, y la latencia acumulada las penaliza. Es la pieza que hace viable la ausencia de nombres: si todo se alcanza navegando desde el objeto inicial, cada paso de la navegación costaría un viaje de ida y vuelta, y sin *pipelining* la tentación sería sustituirlos por rutas y nombres globales. Con *pipelining* no hace falta sacrificar la interfaz, fusionando operaciones para ahorrar viajes —*p. ej.* un método que reciba la ruta completa—, ni la ventaja de poder entregar a cada cliente solo el objeto al que debe llegar. El ejemplo, con una única invocación, no lo necesita.

23.9.4. Niveles del protocolo

La especificación del protocolo se organiza en niveles, y cada implementación declara hasta cuál llega. El primero comprende las referencias a objetos y el *pipelining*. El segundo añade capacidades persistentes: una referencia se puede guardar como un *token* y restaurar más adelante, en otra conexión. El tercero permite interacciones a tres bandas: si A entrega a B una referencia a un objeto alojado en C, B abre una conexión directa con C, en lugar de que sus invocaciones pasen por A. El cuarto permite comprobar que dos capacidades recibidas de partes distintas designan el mismo objeto.

23.10. REST

23.11. Semántica de fallo

Y ¿qué más?

El paso de mensajes y los protocolos petición-respuesta típico de la invocación remota no son el único medio sobre el que se construyen abstracciones de comunicación de alto nivel: las colas de mensajes y el modelo de actores, por ejemplo, resuelven necesidades similares con primitivas asíncronas y sin intentar parecerse a una llamada a función.

